

AULA 4 – OPEN MULTI PROCESSING.

Prof. Rodrigo Gonçalves

www.linkedin.com/in/rodrigouzz



O que é Open Multi-Processing?



Prof. Rodrigo Gonçalves IESB

Open Multi-Processing

→ OpenMP é uma API que permite programar aplicações paralelas em c++ e Fortran usando memória compartilhada.

→ É baseada em diretivas do compilador (pragmas) chamadas de bibliotecas e variáveis de ambiente.

→ Tem por objetivo, simplificar a paralelização de programas sequenciais. Uso deThreads.

OpenMP
Memória compartilhada

→ Open MP, trabalha em um sistema e memória paralela compartilhada. Conhecida como (Shared Memory Systems).

→ Isso que dizer que cada CPU/Core pode ler e escrever em um único espaço lógico de memória.

→ Tem por objetivo, simplificar a paralelização de programas sequenciais sem a necessidade de escrever código complexo para threads. Mas para isso exige sincronização para evitar conflitos.

Threads Programming Models → Considerações

→ Dados compartilhados podem ser acessados por todas as Threads.

→Dados Privados, podem apenas ser acessados apenas pelo owner da Thread.

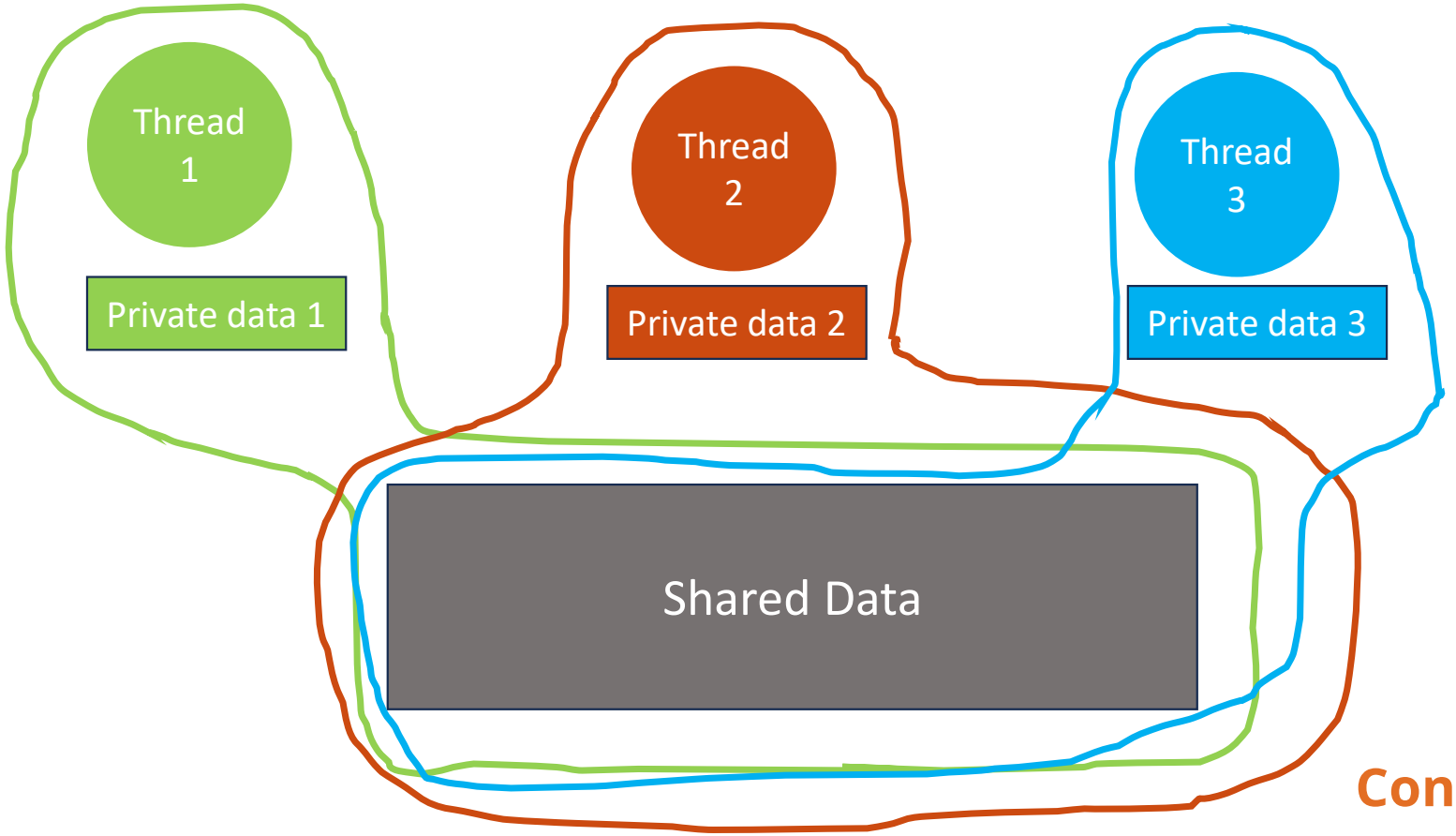
→ Cada threads podem seguir diferentes fluxos de controle para o mesmo programa.

→ Cada thread tem seu próprio controle. (O que deve ser executado em seguida).

→ Geralmente, utiliza-se uma thread por CPU/Core. (pode ser mais cpus/cores). Para usar mais de uma thread por core, a implementação de hardware deve permitir isso. Quem gerencia é SO.



Threads Programming Models → Threads comunicação.



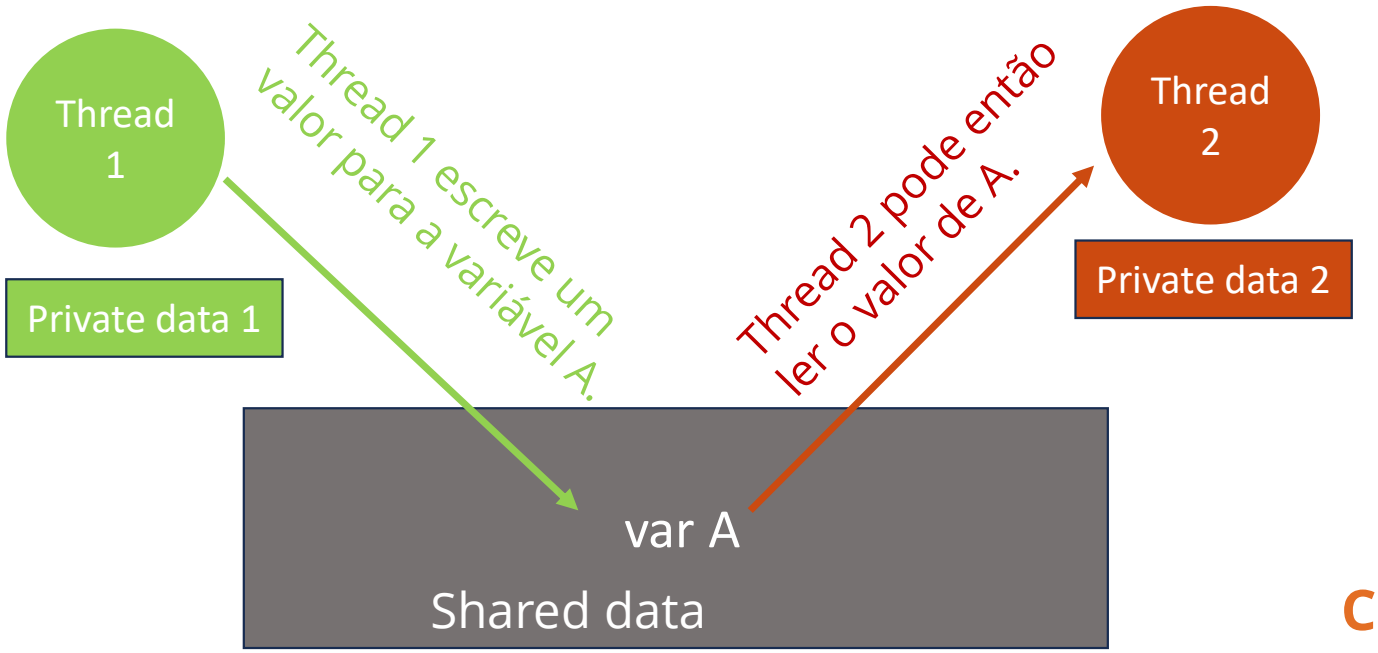


Continua..

Threads Programming Models → Como as threads de comunicam?

→ Para termos paralelismo eficiente, threads devem ter a capacidade de ler dados uns dos outros

→ Threads se comunicam lendo e escrevendo dados compartilhados.



IESB Centro Universitário A Teoria e a Prática Juntas

Continua..

Threads Programming Models → Como as threads de comunicam?

→ Por fim. Threads não há mensagens entre as threads. Elas se comunicam apenas com o compartilhamento de dados, no espaço reservado para isso.

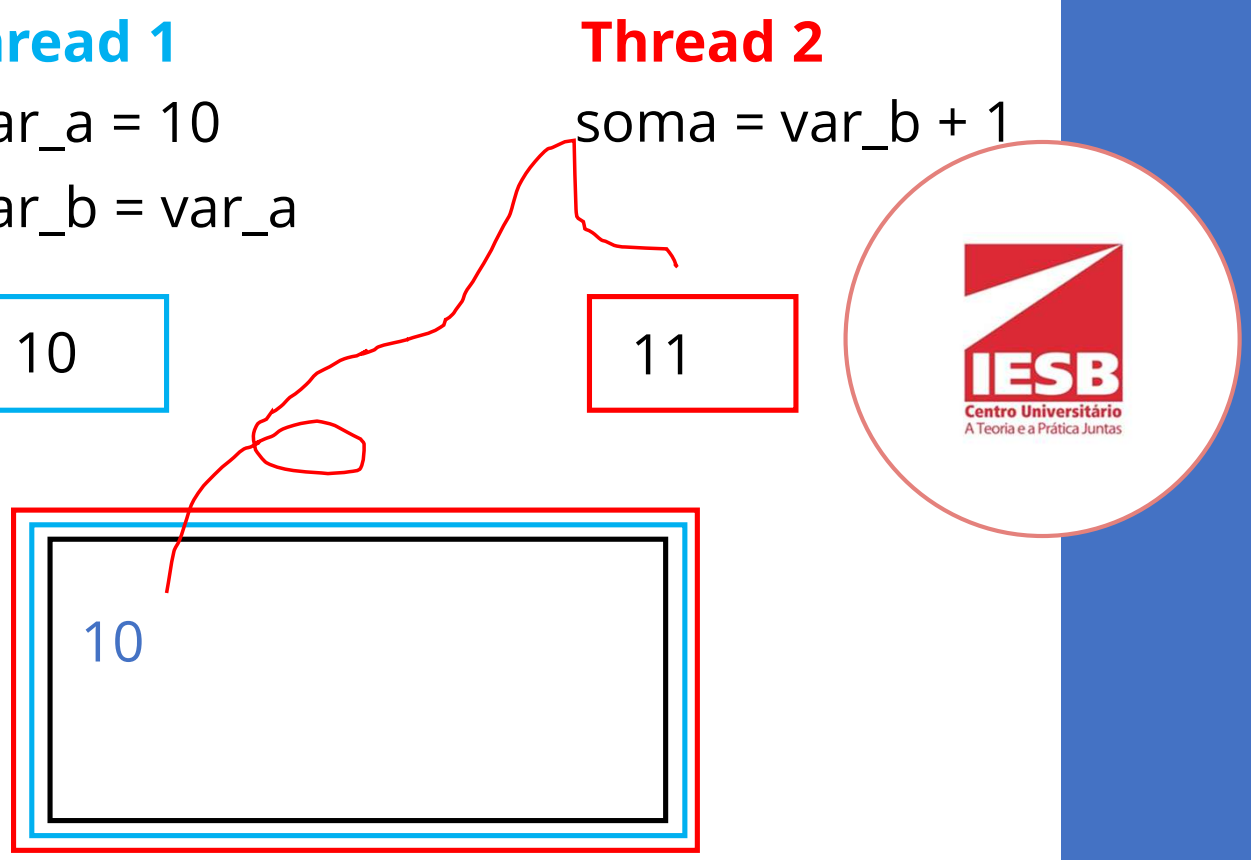


Exemplo de comunicação de Threads.

Program

Private data

Shared data



A comunicação das Threads são assíncronas ?

→ Sim. Por default. E por isso, devemos garantir a programação das threads em uma ordem específica, caso haja a necessidade de compartilhar valores entre threads.

→ Exemplo. A thread 1 deve escrever o valor da variável 1 antes que a thread faça a leitura desse valor.

→ Não há atomicidade em threads. Ou seja, se duas threads tentar atualizar um valor de uma variável compartilhada ao mesmo tempo, isso pode levar a uma sobre escrita dessa variável. Por isso, programar na ordem correta é importante.



Loops Paralelo

→ Loops. São instruções muito utilizadas na programação.

→ Geralmente, loops são independentes. Ou seja, podem ser executados em qualquer ordem do programa.

→ Pensar em loops como Tarefas, que podem ser paralelizáveis.



Vamos para o VSCode !

Exercícios

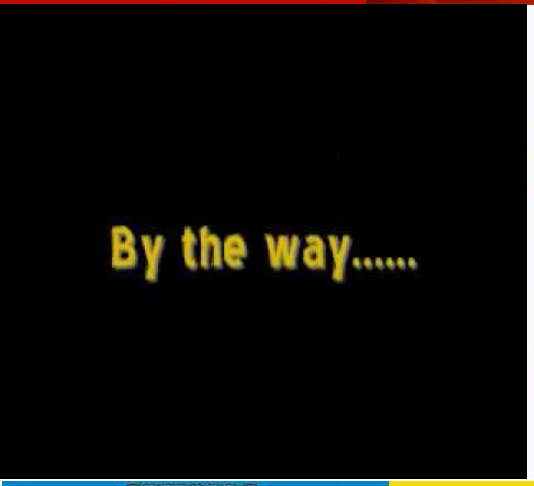



THANKS
FOLKS !!
AND
HOPE YOU
ENJOYED


That's all Folks!


Thank You!


Obrigado
Thank You
Danke
Merci
Grazie
Spasibo
Arigato
Kiitos
Chokrane
Multumesc
Matondo
Mochchakkeram
Dankon
Maake
Welalin
Kia Ora
Vinaka
Mamana
Terma Kasih
Dank Je
Cam on ban
Juspaxar
Nirringrazzjak
Asante
Salamat
Matur Nuwun
Raibh Maith Agat
Ua Tsaug Rau Koj

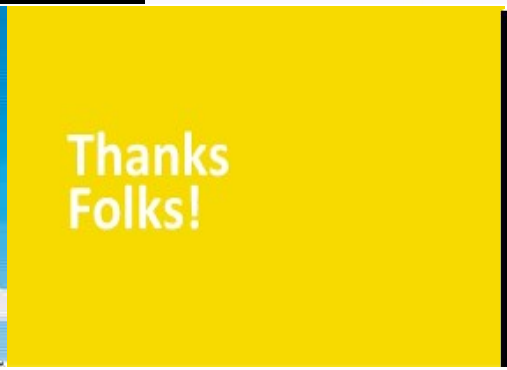

By the way.....


OBRIGADO!


THANK
YOU
GRACIAS
ARIGATO
SHUKURIA
BOLZÏN
MERCI
BÏYAN
DANKSCHEEN
TASHAKKUR ATU
SUKSAMA
MEHRBANI
GRAZIE
JUSPAXAR


OBRIGADO


OBRIGADO
PELA ATENÇÃO


Thanks
Folks!


Thank You!


Thank You !